CohortRound: Sensitivity-Guided Joint Rounding for Fixed-Latency FPGA DSP

Anonymous Authors
Three-Page Research Proposal for DAC

***Abstract*—Fixed-point compilers normally choose word lengths and round every signal independently. This misses a hardware-exploitable degree of freedom: errors introduced on reconvergent DSP paths can cancel at downstream outputs. We propose *CohortRound*, a compiler and FPGA mechanism that groups a few physically local, cycle-aligned quantizers and jointly selects their floor/ceil directions. A bounded-depth selector reads only short discarded-fraction prefixes and minimizes sensitivity-weighted group discrepancy. The compiler co-optimizes cohorts, word lengths, alignment, and selector cost, and rejects cohorts whose control or routing overhead exceeds their datapath savings. An initial four-term prototype exhaustively validates all 4,096 selector states and passes bit-true compiler and RTL tests. Post-route FPGA cost remains the decisive open gate.**

***Index Terms*—FPGA, DSP, fixed point, word-length optimization, joint rounding, compiler**

I. Motivation and Research Question

FPGA word lengths directly affect LUTs, registers, DSP packing, routing, and energy. Existing word-length optimization (WLO) can assign per-variable formats under an output-error constraint [1], and static rounding-mode selection can reduce implementation cost [2]. Nevertheless, the runtime decision at each quantizer is still usually local: round-to-nearest (RTN) minimizes each error separately, not their downstream combination.

Consider quantizers whose outputs later reconverge in a butterfly, FIR/filter bank, matrix–vector product, or convolution reduction. Independent small errors may add with the same sign. Conversely, deliberately choosing a locally non-nearest grid point at one site can cancel errors from other sites. The new question is therefore: *can a compiler synthesize small, fixed-latency rounding cohorts whose controlled error correlation permits narrower FPGA datapaths at the same end-to-end error bound?*

A four-term example illustrates why this differs from local RTN. Suppose the discarded fractions are $(0.49, 0.49, 0.49, 0.49)$. Independent RTN rounds every term down and loses 1.96 output units. A joint policy rounds exactly two terms up, producing a group error of only 0.04. Individual errors become larger at two sites, but the quantity consumed by the DSP reduction becomes much more accurate. This opportunity is common whenever separately quantized values later reconverge. It is absent when errors reach unrelated outputs, have incompatible timing, or require excessive wiring; identifying that boundary is part of the compiler problem rather than an assumption of the method.

Prior filter work shapes roundoff noise by selecting static widths [3], while correlation models estimate noise already created by a datapath [4]. Controlled/dependent rounding provides joint-rounding foundations [5], [6], and DiscQuant applies discrepancy ideas to offline neural-network weights [7]. Stochastic rounding hardware instead makes local PRNG-driven decisions [8]. None directly synthesizes a sensitivity-aware, per-sample joint selector under FPGA pipeline, logic-depth, and physical-locality constraints.

II. CohortRound Mechanism

A. Joint discrepancy

For site i with step Δ_i, write $x_i = \Delta_i(n_i + a_i)$, where $n_i = \lfloor x_i/\Delta_i \rfloor$ and $a_i \in [0, 1)$. A legal floor/ceil decision $r_i \in \{0, 1\}$ introduces

$$\delta_i = \Delta_i(r_i - a_i). \tag{1}$$

For a cohort C in an affine downstream region, its monitored-output error is

$$\boldsymbol{e}_C = A_C D_C(\boldsymbol{r}_C - \boldsymbol{a}_C), \quad D_C = \mathrm{diag}(\Delta_i), \tag{2}$$

where columns of A_C are downstream sensitivities. Instead of independent RTN, an ideal cohort selects

$$\boldsymbol{r}^*_C = \arg \min_{\boldsymbol{r}\in\{0,1\}^{|C|}} \|W A_C D_C(\boldsymbol{r} - \boldsymbol{a}_C)\|_\infty. \tag{3}$$

The compiler initially admits exact affine regions; nonlinear regions require a sound Jacobian/remainder envelope rather than sampled sensitivities.

B. Prefix selector and certificate

Sending every discarded bit to a shared unit would erase the saving. The selector therefore observes only b leading residue bits $p_i = \lfloor 2^b a_i \rfloor$ and implements a small truth table $T_C(\boldsymbol{p}) \in \{0, 1\}^{|C|}$. Its deterministic certificate is

$$\Gamma_C = \max_{\boldsymbol{p}} \sup_{\boldsymbol{a}\in\mathcal{B}(\boldsymbol{p})} \|W A_C D_C(T_C(\boldsymbol{p}) - \boldsymbol{a})\|_\infty. \tag{4}$$

For k equal-sensitivity terms with common step Δ, choose $m = \mathrm{round}(\sum_i a_i)$ and any binary vector with $\sum_i r_i = m$. Then the group error is at most $\Delta/2$, whereas independent RTN can approach $k\Delta/2$. With prefix midpoints $\tilde{a}_i = (p_i + 1/2)2^{-b}$, the bound becomes

$$|e_C| \le \Delta \left(\frac{1}{2} + \frac{k}{2^{b+1}}\right). \tag{5}$$

Thus the implemented $k = 4, b = 3$ selector has a 0.75Δ bound while using only twelve residue bits and a three-bit correction count.

For unequal sensitivities, the identity of the rounded-up lanes matters, not only their count. HCDR enumerates each prefix cell and solves the robust local problem

$$T_C(\boldsymbol{p}) = \arg \min_{\boldsymbol{r}\in\{0,1\}^{|C|}} \sup_{\boldsymbol{a}\in\mathcal{B}(\boldsymbol{p})} \|WA_C D_C(\boldsymbol{r}-\boldsymbol{a})\|_\infty. \quad (6)$$

The resulting mapping is minimized into LUTs, comparators, and a small adder tree. Prefix width is itself a design variable: increasing b tightens the certificate but adds source fanout, wires, and selector inputs. Candidate selectors that miss the cycle budget are either pipelined with explicitly charged alignment registers or rejected.

C. Hardware-Constrained Discrepancy Rounding

Our compiler algorithm, HCDR, performs four steps. First, it extracts site scales, sensitivities, lifetimes, pipeline stages, and placement regions from a scheduled DSP graph. Second, it constructs a candidate-cohort hypergraph and uses a software joint-rounding oracle to estimate attainable bit savings. Third, exact truth-table/MILP search synthesizes small prefix selectors subject to depth, fanout, and span limits; larger groups use a sensitivity-aware heuristic measured against the exact small-instance oracle. Finally, a hardware-aware set-packing pass selects non-overlapping cohorts jointly with word lengths and schedule alignment:

$$\begin{aligned}\min_{\boldsymbol{f},\boldsymbol{z},\{T_C\},S} \quad & C_{\text{data}}(\boldsymbol{f},S) + \sum_C z_C(C_{\text{sel}} + C_{\text{route}}) \\ \text{s.t.} \quad & \Gamma_{\text{single}} + \sum_C z_C \Gamma_C \le \epsilon, \quad \text{II} \le \text{II}_{\max}. \end{aligned} \quad (7)$$

Timing, alignment, overflow, and one-cohort-per-site constraints are also enforced. This explicit cost model lets the compiler reject unprofitable cohorts rather than assuming joint rounding is always beneficial.

D. What is genuinely new

Joint rounding by itself is not new. The proposed contribution is the combined compiler/hardware object

$$\textit{cohort} = (C, A_C, D_C, b, T_C, \Gamma_C, \textit{placement window}), \quad (8)$$

which connects compile-time sensitivities and physical constraints to a per-sample hardware policy. WLO changes static formats but not the correlated runtime decisions. Dependent rounding supplies mathematical tools but does not map dynamic DSP intermediates onto a bounded-latency circuit. Correlation analysis predicts existing noise but does not synthesize its signs. DiscQuant uses discrepancy to round static model weights offline. Stochastic rounding is runtime, but each decision is PRNG-driven rather than jointly conditioned on downstream sensitivities.

The distinction from delayed rounding is also essential. If all cohort members feed one immediate sum with equal sensitivity, retaining guard bits and rounding once is a strong baseline and may be preferable. CohortRound becomes structurally distinct when values are quantized before traversing different branches, buffers, or operators and later affect one or more outputs with unequal signed sensitivities. In such graphs, moving one common rounding point to the end is not equivalent without preserving wide data across the intervening region. The evaluation therefore includes both direct reductions and branched/reconvergent graphs; success on the former alone is insufficient.

TABLE I
COMPLETED PRE-VIVADO GATES.

Gate	Result	Coverage
Prefix oracle	4,096/4,096	all 8^4 states
Bit-true datapath	2,000/2,000	random signed MACs
Compiler tests	7/7	checker, lowering, fail-closed
Chisel/Verilator	5/5	target and two baselines
RTL lint	3/3	all generated tops

III. PROTOTYPE EVIDENCE AND DAC EVALUATION

We added a `cohort_mac_reduce` intrinsic, a fail-closed canonical dot-product recognizer, bit-true interpreter semantics, Chisel lowering, and three matched RTL designs. All use 24 signed $\text{Fixed}[16, 14]$ terms, the same digit-serial multiplier and control schedule, and a $\text{Fixed}[20, 13]$ output. Only the rounding/reduction policy changes: CohortRound, independent per-product RNE, or full-precision accumulation followed by one RNE.

The current target forms exact products sequentially. For every group of four, it arithmetic-shifts each signed product to a 17-bit floor, collects the three most significant discarded bits, and accumulates the floors in 19 bits. The selector converts the four prefixes into a correction in $[0, 4]$, which is added to a 23-bit global accumulator. Negative products use the same floor–residue decomposition, so no sign-specific approximation is hidden in the software model. Two matched controls preserve input/output formats, operation count, multiplier, handshake, and schedule: one independently rounds every product, while the other accumulates in 38 bits and rounds once.

The lowering reduces the global accumulator from 38 to 23 bits and uses a 19-bit four-term group accumulator. However, the present serial multiplier still forms a full 32-bit product; therefore these results establish numerical and RTL feasibility, not a large total-area saving.

The tests deliberately cover more than average error. The Python selector exhausts all prefix states, including ties; an independently written integer oracle checks negative operands, extreme products, output wraparound, and 2,000 random vectors. RTL tests repeat exhaustive selector checking, verify the one-cycle valid/bubble protocol, and cross-check the complete 24-term machine and both baselines. Unsupported cohort sizes and noncanonical kernels fail closed instead of silently falling back to different arithmetic.

The full evaluation will cover FIR/polyphase filters, FFT butterflies, matrix–vector/beamforming, and convolution reductions, with negative controls that have no local alignable cohort. Baselines include truncation, independent RTN and stochastic rounding, static mode optimization [2], and width-only WLO [1]. We will report post-route LUT/FF/DSP/BRAM, Fmax, II, latency, energy/sample, routing/alignment overhead, error certificates, and exact-oracle gap. The idea passes only if at least three basic DSP kernels save at least one fractional bit at the same deterministic error bound, retain II, lose less than 5% Fmax, and achieve net post-route area or energy benefit. If the shared multiplier dominates, a truncated-product/high-product-plus-prefix lowering is required; changing the device or timing target is not evidence.

To isolate causality, the study uses four ablations: (i) identical widths with independent decisions, measuring the value of correlation; (ii) oracle cohorts without locality constraints, measuring the physical feasibility gap; (iii) identical cohorts with full residues versus short prefixes, measuring the information–routing trade-off; and (iv) compiler-selected versus random or cardinality-matched cohorts, measuring the value of sensitivity analysis. Each design is synthesized under the same part, clock, II, I/O, and multiplier architecture. Power claims require workload-derived SAIF/VCD activity rather than vectorless estimates.

Three failure modes are reported explicitly. First, many kernels may expose only isolated quantizers or sensitivities that do not permit cancellation. Second, the selector and alignment network may consume more resources than one saved fractional bit. Third, a serial architecture may be multiplier-dominated, making reduction-width savings invisible at top level. The last case motivates a high-product-plus-prefix multiplier, but that extension must be evaluated as a separate mechanism and ablation rather than credited to cohort rounding.

A. Compiler integration and supported scope

The front end first lowers arithmetic casts and explicit quantization sites into an IR that records source/destination formats, discarded-bit ranges, and overflow semantics. A backward sensitivity pass annotates each site with its transfer vector to selected outputs. Scheduling then supplies availability cycles and lifetimes, while a placement proxy supplies candidate regions and wire-distance estimates. HCDR consumes these annotations and emits ordinary fixed-width operators plus a selector module, residue-prefix connections, and any required delay registers. The bit-true interpreter evaluates exactly the same prefix policy used by RTL, preventing a floating-point software oracle from silently defining different tie or signed-shift behavior.

The first compiler version deliberately targets real-valued affine regions and acyclic reductions. Saturation and overflow are orthogonal: range analysis must prove no overflow or model the selected wrap/saturate semantics before a rounding certificate is valid. Feedback filters require an invariant over both data state and rounding state and are therefore a separate extension. Likewise, the stateless selector provides a per-invocation deterministic discrepancy bound; it does not claim probabilistic unbiasedness. A future debt controller may reduce long-term drift only if finite-state reachability proves its debt is bounded for every legal input sequence.

The expected deliverables are consequently concrete: a cohort-aware numerical IR and discovery pass; an exact small-cohort robust selector algorithm with a scalable heuristic; machine-checkable error certificates that include unseen residue suffixes; and an RTL backend with post-route cost feedback. Releasing rejected candidates and their causes is also important: it reveals whether the limiting factor is numerical structure, schedule alignment, selector logic, or routing, and prevents reporting only favorable kernels.

IV. Conclusion

CohortRound turns runtime correlation among quantization errors into a compiler-visible FPGA design variable. Its novelty is not discrepancy rounding alone, but sensitivity-aware cohort discovery, fixed-latency prefix-selector synthesis, deterministic certification, and physical-cost-aware extraction. The completed prototype supports the numerical and implementation mechanism; matched Vivado post-route results remain the decisive test of DAC-level impact.

References

[1] I. Winandy, A. Dion, F. Manni, P.-L. Garoche, D. B. Khalifa, and M. Martel, “Automated fixed-point precision optimization for FPGA synthesis,” *IEEE Open Journal of Circuits and Systems*, vol. 6, pp. 192–204, 2025.

[2] D. Menard, D. Novo, R. Rocher, F. Catthoor, and O. Sentieys, “Quantization mode opportunities in fixed-point system design,” in *Proceedings of the 18th European Signal Processing Conference (EUSIPCO)*, Aalborg, Denmark, 2010, pp. 542–546.

[3] G. A. Constantinides, P. Y. K. Cheung, and W. Luk, “Roundoff-noise shaping in filter design,” in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, vol. 4, 2000, pp. 57–60.

[4] J.-C. Naud, D. Menard, G. Caffarena, and O. Sentieys, “A discrete model for correlation between quantization noises,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 59, no. 11, pp. 800–804, 2012.

[5] L. H. Cox and L. R. Ernst, “Controlled rounding,” *INFOR: Information Systems and Operational Research*, vol. 20, no. 4, pp. 423–432, 1982.

[6] R. Gandhi, S. Khuller, S. Parthasarathy, and A. Srinivasan, “Dependent rounding and its applications to approximation algorithms,” *Journal of the ACM*, vol. 53, no. 3, pp. 324–360, 2006.

[7] J. Chee, A. Backurs, R. Heck, L. Zhang, J. Kulkarni, T. Rothvoss, and S. Gopi, “Discquant: A quantization method for neural networks inspired by discrepancy theory,” in *Proceedings of the 38th Conference on Learning Theory*, ser. Proceedings of Machine Learning Research, vol. 291. PMLR, 2025, pp. 924–951. [Online]. Available: https://proceedings.mlr.press/v291/chee25a.html

[8] M. Mikaitis, “Stochastic rounding: Algorithms and hardware accelerator,” in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2021.